

STACK

Stack

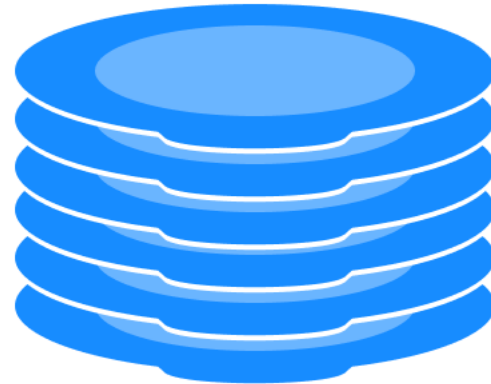
- Stack is a non primitive and linear data structure.
- In a stack, insertion of new data item and deletion of already existing data item is done from only one end is known as top of stack.

Cont...

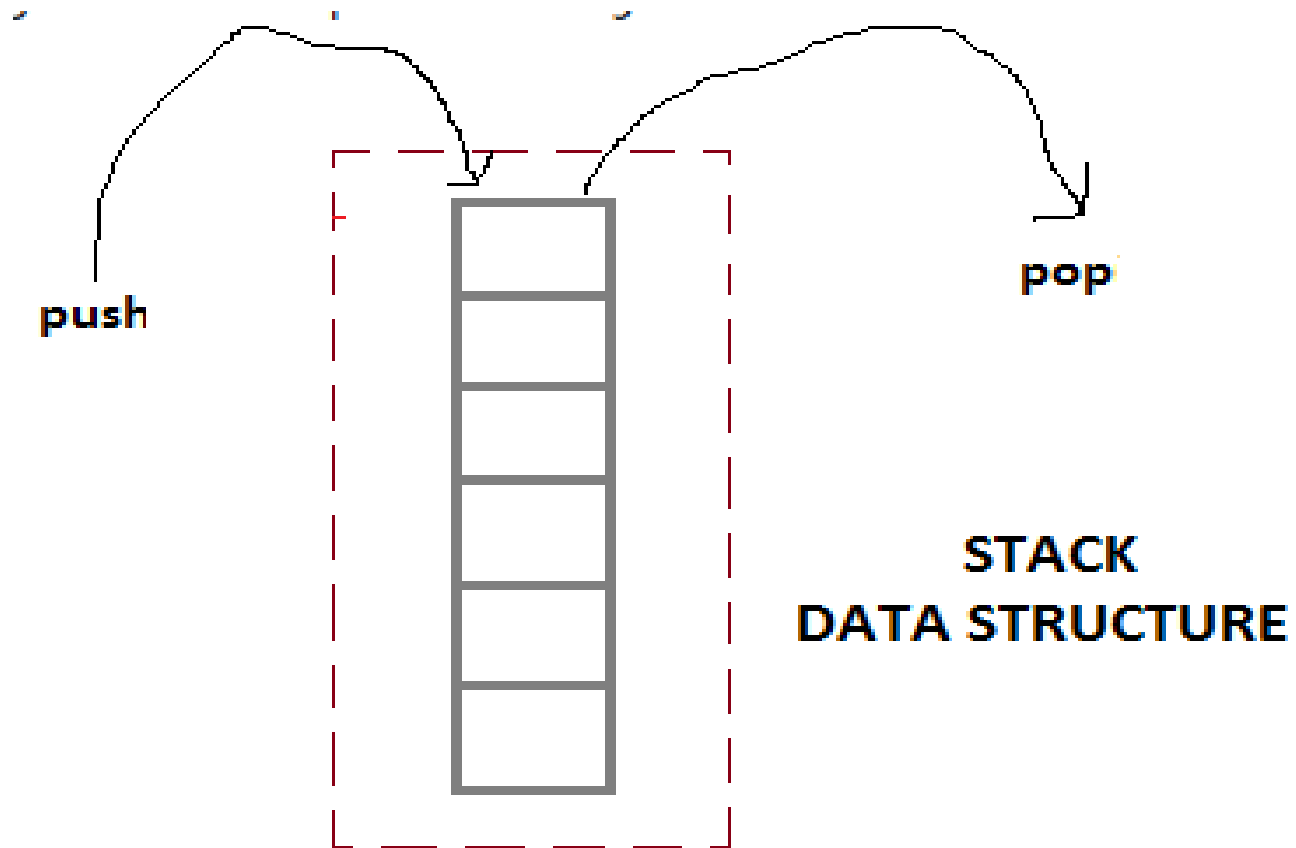
- Due to this property, stack will be work on Last in First out (LIFO) so stack is called LIFO data structure.
- LIFO means last added item in a stack will be deleted first.

Example

- A common model of a stack is plates in a marriage party. Fresh plates are pushed on the top and popped off from the top. It is just like a pile of plates kept on top of each other.

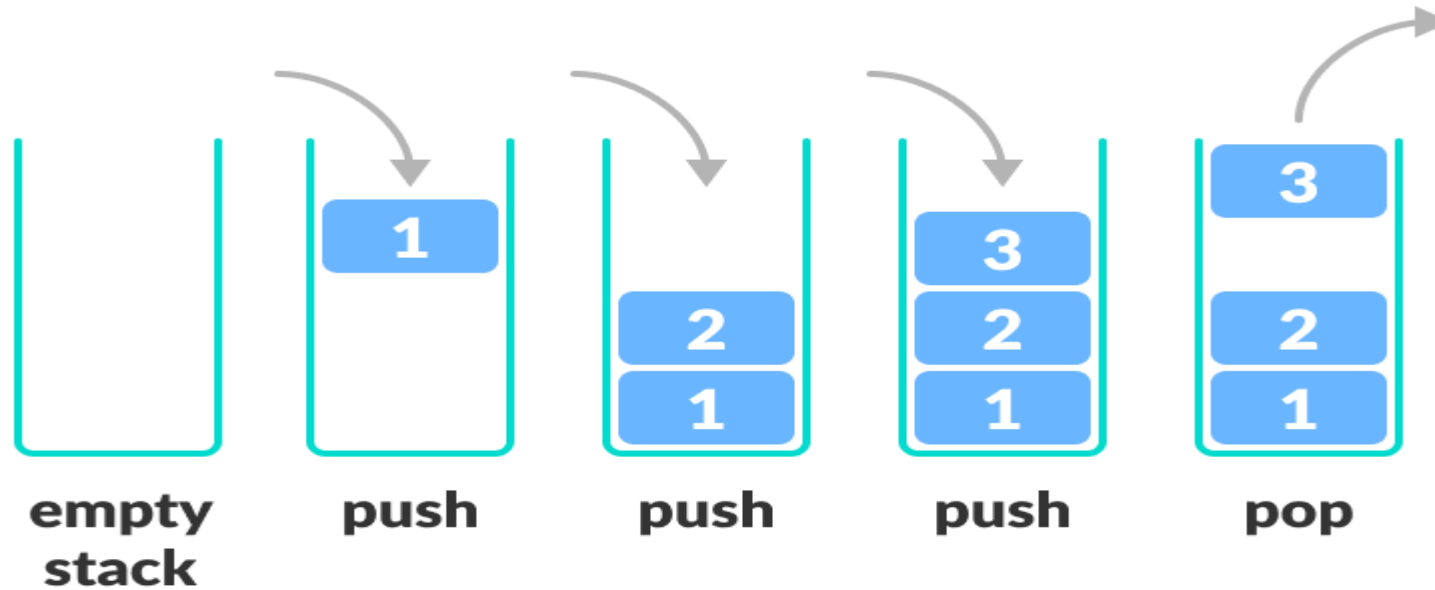


Stack representation similar to a pile of plate



LIFO Principle of Stack

- In programming terms, putting an item on top of the stack is called **push** and removing an item is called **pop**.



Cont...

- In the above image, although item 3 was kept last, it was removed first
- so it follows **the Last In First Out(LIFO)** principle.

Basic Operations of Stack

- A stack allows the following operations.
 - **Push**: Add an element to the top of a stack.
 - **Pop**: Remove an element from the top of a stack.
 - **IsEmpty**: Check if the stack is empty.
 - **IsFull**: Check if the stack is full.
 - **Peek**: Get the value of the top element without removing it

Implementation of Stack

- There are two way for implementing a stack.
 - Static implementation (By using Array)
 - Dynamic implementation (By using LinkedList)

1. Static implementation (By Using Array)

- Here For static implementation of a stack, we will be used array.

Working of Stack Data Structure

- A variable called TOP is used and it will contain the index of top most element in a stack.
- Initially stack is empty and value of $TOP = -1$.
- On pushing (inserting) an element in a stack, Firstly we increase the value of TOP by one i.e. $TOP = TOP + 1$ then add new element in the position pointed to by TOP.

Cont...

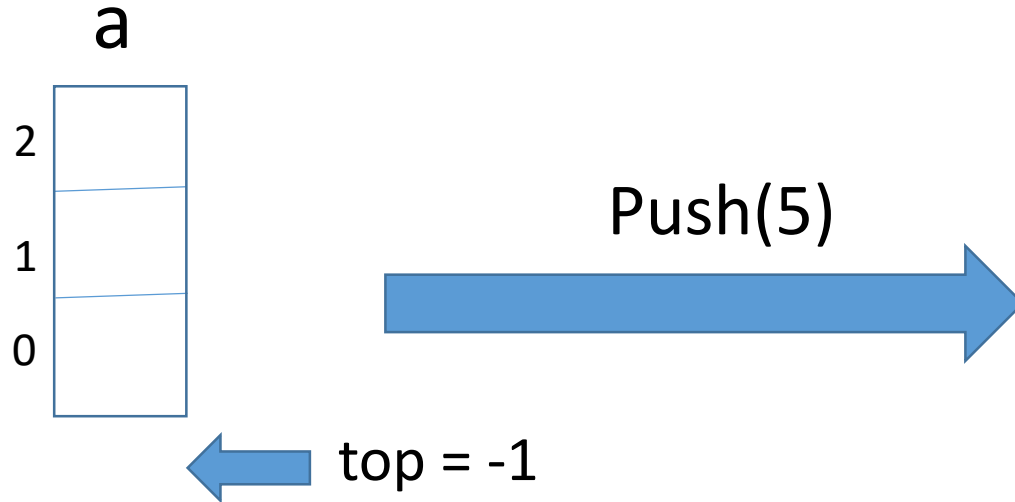
- On popping (deleting) an element from a stack, we reduce the value of TOP by 1 i.e. $TOP = TOP - 1$.
- Before pushing, we check if the stack is already full.
- Before popping, we check if the stack is already empty.
-

Example of push and pop operation

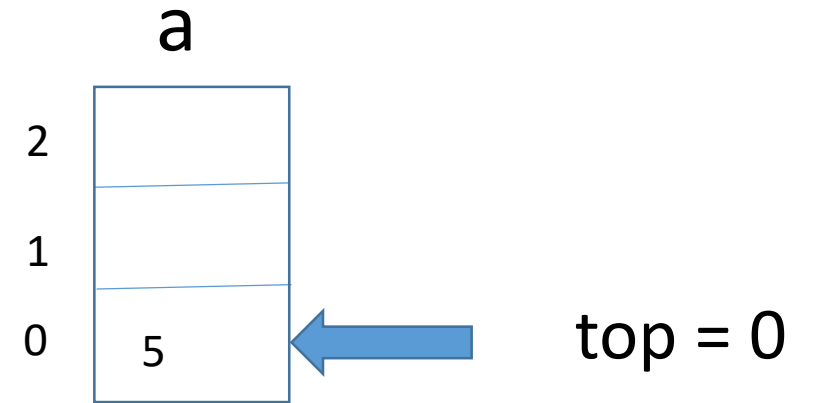
- Suppose there is an array `a` of size 3 i.e. `int a[3];`

Example of Push operation

Step-1



Initially stack is empty so
 $\text{top} = -1$



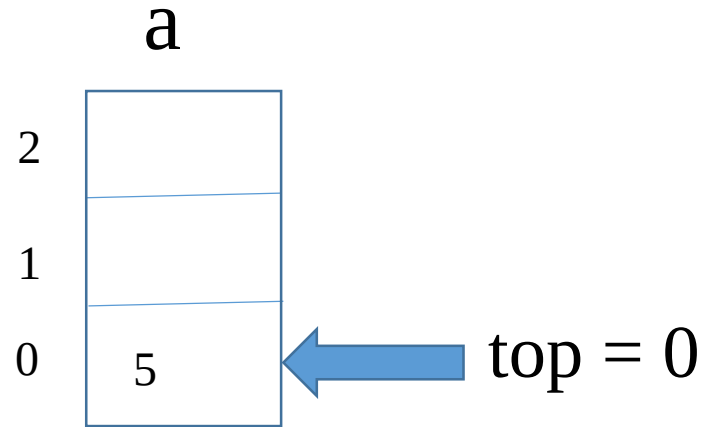
$$\text{top} = \text{top} + 1 = -1 + 1 = 0$$

$$\text{top} = 0$$

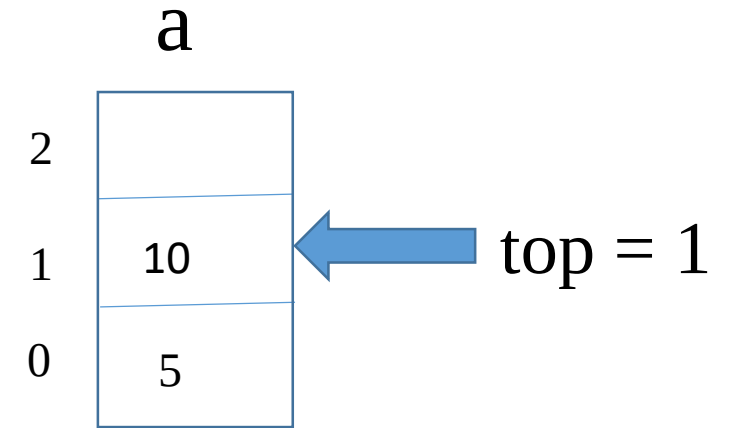
$$a[\text{top}] = a[0] = 5$$

Cont...

Step-2



Push(10)



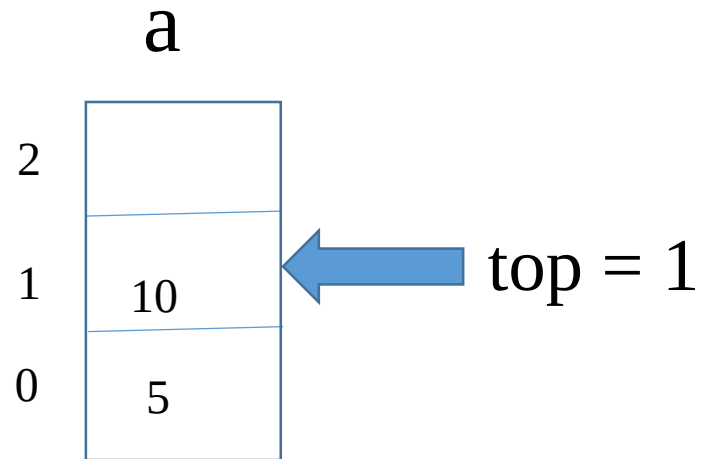
$\text{top} = \text{top} + 1 = 0 + 1 = 1$

$\text{top} = 1$

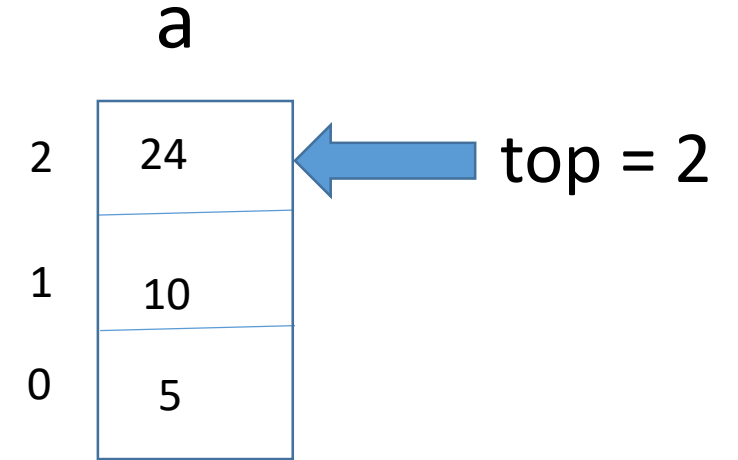
$\text{a}[\text{top}] = \text{a}[1] = 10$

Cont...

Step-3



Push(24)



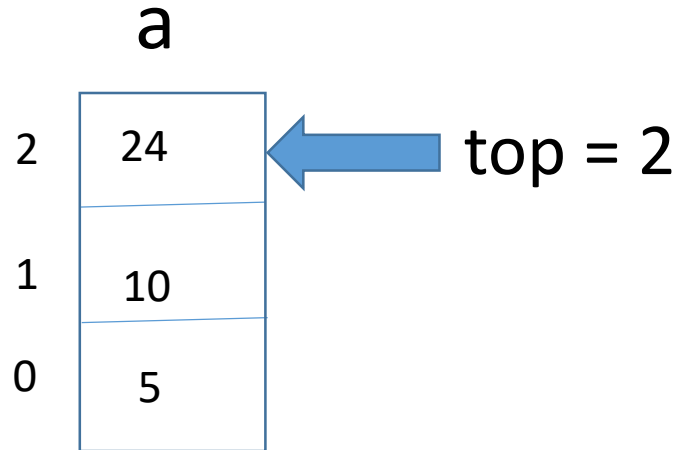
$\text{top} = \text{top} + 1 = 1 + 1 = 2$

$\text{top} = 2$

$a[\text{top}] = a[2] = 24$

Cont...

Step-4



Push(30)

As, stack is full, so we can not push (insert) element 30 and this condition is called stack overflow condition.

Note: Stack overflow condition is $\text{top} == \text{maxsize} - 1$

Where

maxsize = maximum size of an array.

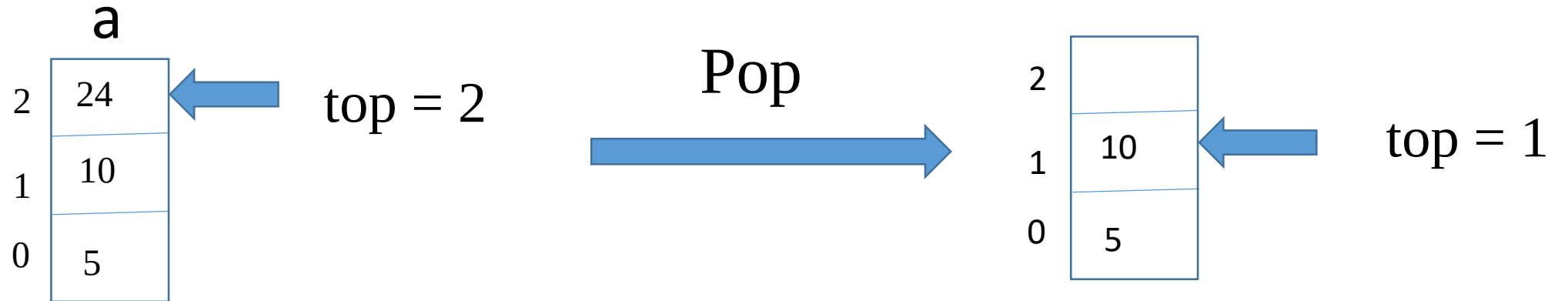
Here: $\text{top} = 2$, $\text{maxsize} = 3$.

$\text{maxsize} - 1 = 3 - 1 = 2$

$2 == 2$, it means stack is full and we can not push (insert) element 30 and this condition is called stack overflow condition.

Example of Pop operation

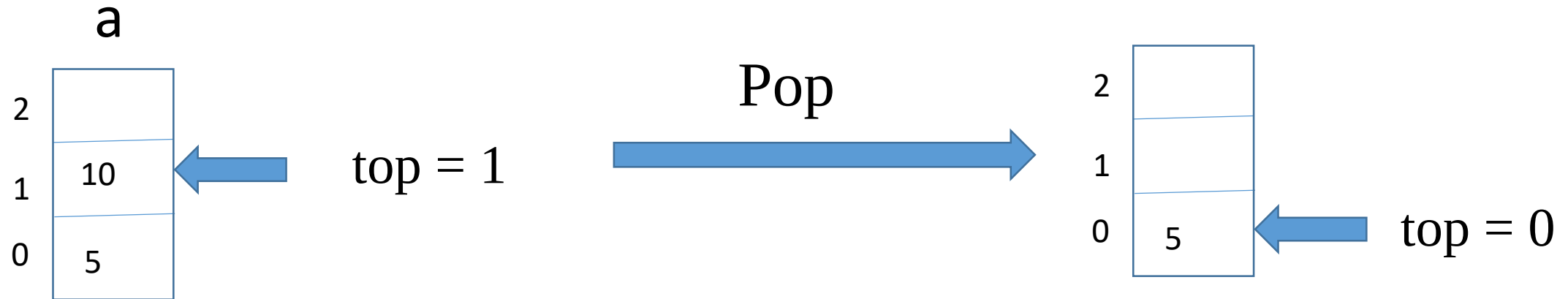
Step-1



$$\text{top} = \text{top} - 1 = 2 - 1 = 1$$
$$\text{top} = 1$$

Cont...

Step-2

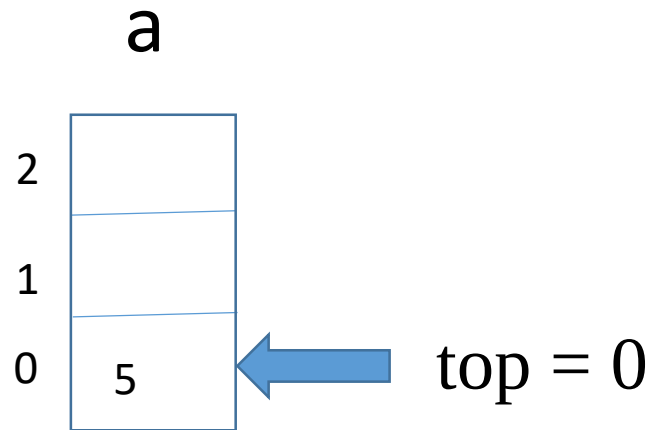


$$\text{top} = \text{top} - 1 = 1 - 1 = 0$$

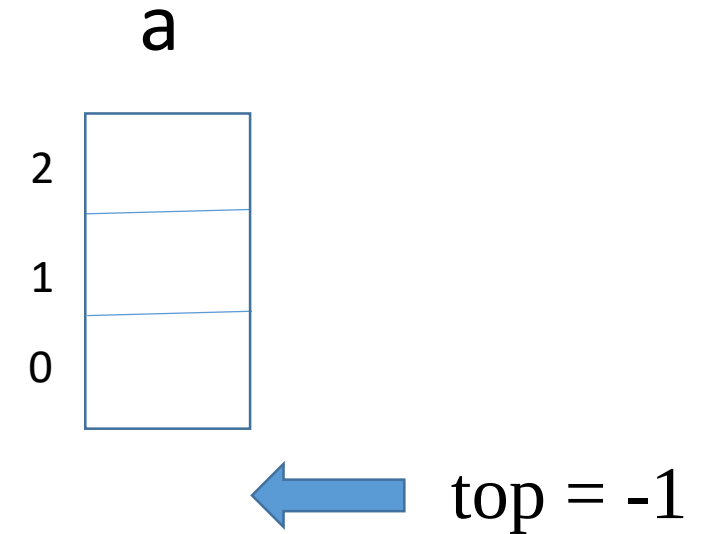
$$\text{top} = 0$$

Cont...

Step-3



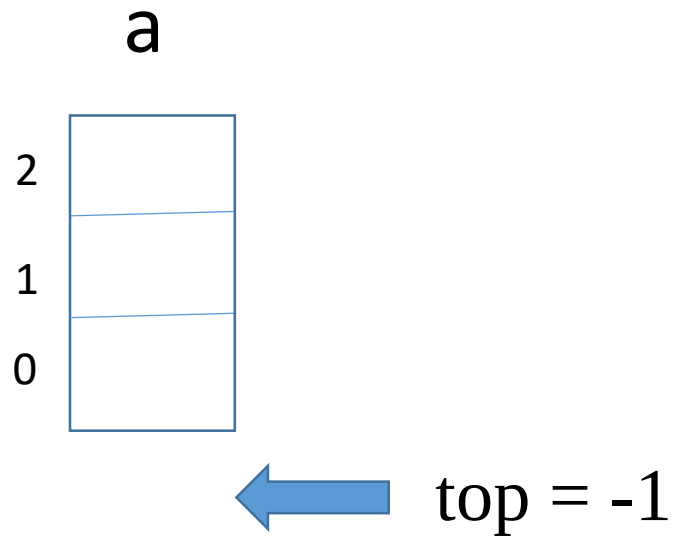
Pop



$\text{top} = \text{top} - 1 = 0 - 1 = -1$
 $\text{top} = -1$

Cont...

Step-4



Pop



Now, we can not pop (delete) a element from a top of a stack because stack is empty and this condition is called **stack under flow condition**.

Note: Stack under flow condition is $top == -1$

Operation on Stack

- There are mainly two operation of stack.
 - Push operation
 - Pop operation

Push operation

- The process of adding a new element to the top of stack is called push operation.
- After every pushing (inserting) operation, firstly the value of top will increase by one then element will be inserted to the top of a stack.

Cont...

- In case stack (array) is full and no new element can be added. It is called stack full (stack overflow) condition.
- Condition for stack overflow is:

$top == maxsize - 1$

Where $maxsize$ = maximum size of an array.

Algorithm for push (insert) operation on a stack

`push(s[maxsize],item,top)`

Let `s[maxsize]` is an array which name is `s` and `maxsize` is the maximum size of an array `s`. `top` is a variable that will contain the index of top most element of a stack.

This algo push (insert) the element **item** at the **top** of stack `s`.

Step 1: Start

Step 2: if (top == maxsize -1)
 {
 printf(“stack is overflow”);
 }
else
 {
 set top = top + 1;
 set s[top] = item;
 }

Step 3: Exit

Pop operation

- The process of deleting an element from the top of stack is called pop operation.
- After every pop (delete) operation, the value of top will decrease by one.

Cont...

- If there is no element in a stack and pop operation is performed then this condition is called stack underflow.
- Condition for stack underflow is:
$$\text{top} = -1$$

Algorithm for pop (delete) operation on a stack

`pop(s[maxsize], top)`

Let `s[maxsize]` is an array which name is `s` and `maxsize` is the maximum size of an array `s`. `top` is a variable that will contain the index of top most element of a stack.

This algo pop (delete) the element **from** the **top** of stack `s`.

Step 1: Start

Step 2: if (top == -1)

{

printf("stack is underflow");

}

else

{

set top = top - 1;

}

Step 3: Exit

Application of Stack in Data Structures

- Expression Evaluation and Conversion
- Backtracking
- Function Call
- Parentheses Checking
 - For e.g $((a + b) * (c + d))$ is valid but $\{\{a+b\}\} * (b+d]$ is not valid.
- String Reversal
- Memory Management

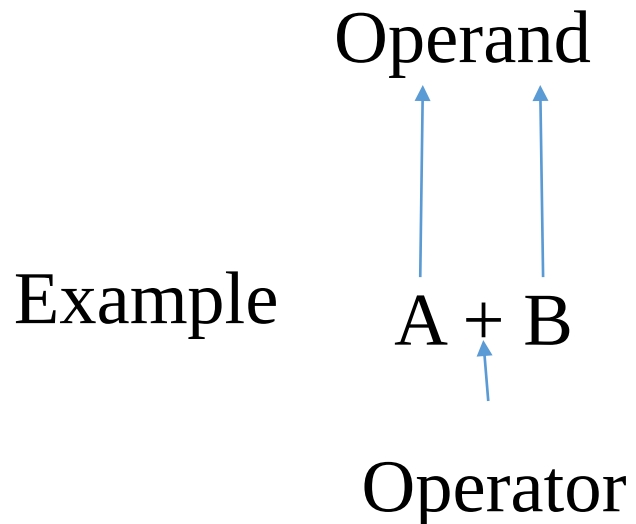
Notations

- There are 3 types of notation.
 - Infix notation.
 - Prefix notation.
 - Postfix notation.
- An arithmetic expression can be represented in various form such as Infix form , prefix form and postfix form .

1. Infix notation (Infix form)

- In infix notation, operator is placed in middle of its operands.
- When the operators exist b/w two operands then the expression is called **infix polish notation**.

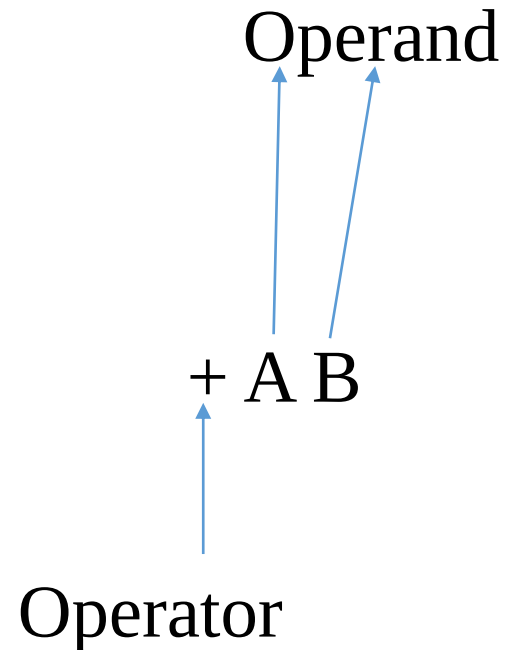
For ex: $a + b * c$



2. Prefix notation (Prefix form)

- In prefix notation, operator is placed before its two operands. It is also called polish notation.

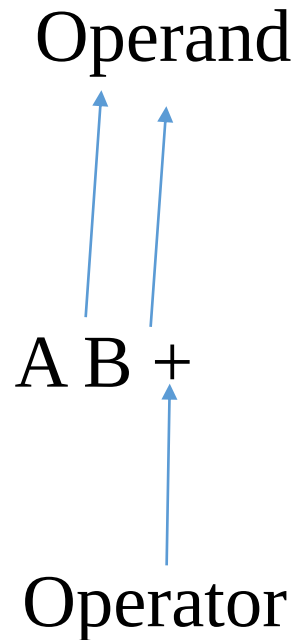
Example



3. Postfix notation (Postfix form)

- In postfix notation, operator is placed after its two operands. It is also called reverse polish notation or suffix notation.

- Example :



Postfix:

When the operators come after their operands the resulting expression is called the reverse polish notation or postfix expression.

For Ex: $a+b*c$ infix form

Step 1: $a+(b*c)$ parenthesized expression

Step 2: $a+(bc*)$ convert the multiplication

Step 3: $a(bc*)+$ convert the addition

Step 4: $abc*+$ postfix form

For Ex-1: Convert the expression $A*B+C/D$ to postfix form.

Solution:

$A*B + C/D$

$(A*B) + C/D$

$(T) + C/D$

$T=AB^*$

$(T) + (C/D)$

$(T)+ (CD/)$

$(T) + (S/)$

$S=CD/$

$T + S$

$TS+$

$AB^*CD/+$

Postfix expression

For Ex-2: Convert the expression $(A+B) * C/D + E^{\wedge} F/G$ to postfix expression.

Solution :

$$(A+B) * C/D + E^{\wedge} F/G$$

$$(AB+)^* C/D + (E^{\wedge}F)/G$$

$$T * C/D + (EF^{\wedge})/G$$

$$T * C/D + S/G$$

$$(T*C)/D + S/G$$

$$(TC^*)/D + S/G$$

$$Q/D + S/G$$

$$(Q/D)+ S/G$$

$$(QD/)+S/G$$

$$P + (S/G)$$

$$P+ (SG/)$$

$$P+ O$$

$$PO+$$

($\wedge$ has the higher priority)

$$T=AB+$$

$$S=EF^{\wedge}$$

$$Q=TC^*$$

$$P=QD/$$

$$O=SG/$$

Now expand the expression $PO+$

$$PO+$$

$$QD/O+$$

$$TC^*D/O+$$

$$AB+C^*D/O+$$

$$AB+C^*D/SG/+$$

$$AB+C^*D/EF^{\wedge}G/+ \quad \text{post fix operation}$$

Algorithm for converting infix expression to postfix form:

POSTFIX (Q, P)

Q : Infix Expression

P : Equivalent postfix expression

Step 1: push "(" on to STACK and add ")" to the end of Q

Step 2: Scan Q from left to right and repeat step 3 to 6 for each element of Q until the STACK is empty

Step 3: if an operand is encountered, add it to P.

Step 4: if a left parenthesis is encountered, push it on to STACK.

Step 5: if an operator (★) is encountered, then :

- Add operator (★) to stack

[End of if structure]

- Repeatedly pop from the STACK (on the top of STACK) and add each operator to P, which has the same precedence as or higher precedence than operator (★).

Step 6: if a right parenthesis is encountered, then:

- Repeatedly pop from STACK (on the top of STACK) and add each operator to P until a left parenthesis is encountered, then:
- Remove the left parenthesis [do not add the left parenthesis to P.]

Step 7: Exit

For Ex: Stack Implementation of infix expression $z + (y * x - (w / v ^ u) * t) * s$ to postfix

Label Number	Symbol Scanned	Stack	Expression
1	z	(	z
2	+	(+	z
3	(	(+ (	z
4	y	(+ (	z y
5	*	(+ (*	z y
6	x	(+ (*	z y x
7	-	(+ (-	z y x *
8	(	(+ (- (	z y x *
9	w	(+ (- (	z y x * w
10	/	(+ (- (/	z y x * w
11	v	(+ (- (/	z y x * w v
12	^	(+ (- (/ ^	z y x * w v
13	u	(+ (- (/ ^	z y x * w v u
14	)	(+ (-	z y x * w v u ^ /
15	*	(+ (- *	z y x * w v u ^ /
16	t	(+ (- *	z y x * w v u ^ / t
17	)	(+	z y x * w v u ^ / t * -
18	*	(+ *	z y x * w v u ^ / t * -
19	s	(+ *	z y x * w v u ^ / t * - s
20	)		z y x * w v u ^ / t * - s * +

Precedence

Precedence

Bracket () (Higher Precedence)

Exponential operator (^)

Multiplication / Division (* , /)

Addition / Subtraction (+, -) (Lower Precedence)



Converting infix expression into postfix expression

Q. Converting following infix expression $a+(b*c-(d/e)*g)*h$ into postfix expression.

Ans: **$abc*de/g*-h*+$**

$$a+(b*c-(d/e)*g)*h$$

Cont...

Q: Converting $a+(b*c)$ expression into postfix expression.

Ans: $abc*+$

$$a+(b*c)$$

Converting infix expression into prefix expression

Q. Converting following infix expression $a+(b*c-(d/e)*g)*h$ into prefix expression.

$$a+(b*c-(d/e)*g)*h$$

Cont...

Q: Converting $a+(b*c)$ expression into prefix expression.

$$a+(b*c)$$

Evaluation of the Postfix Expression:

Suppose, E is an arithmetic expression written in polish postfix notation.

Exp. **2 3 10 + * 8 2 / -**

The equivalent infix expression is

$$2*(3+10) - 8/2$$

Exp : **2 3 10 + * 8 2 / -**

Label : 1 2 3 4 5 6 7 8 9

Table: Evaluation of expression with the help of Stack

Label Number	Symbol Scanned	Stack Contents
1	2	2
2	3	2 3
3	10	2 3 10
4	+	2 13
5	*	26
6	8	26 8
7	2	26 8 2
8	/	26 4
9	-	22

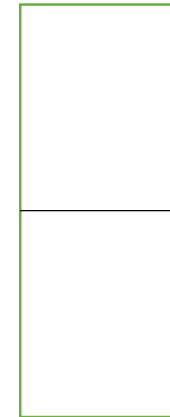
4. Evaluate a postfix expression

Q. Evaluate the postfix expression $AB+C*D/$ if $A=2$, $B=3$, $C=4$ and $D=5$ by using stack.

Ans: $AB+C*D/$

Step 1 : initially stack is empty.

Stack



Initially Stack is empty.

Cont...

Step 2 : First element is operand A so push A into stack.

Stack



Cont...

Step 3 : Second element is also operand B so push B into stack.

Stack



Cont...

Step 4 : Third element is operator + so pop top 2 operands from the stack i.e B and A.

Stack



Cont...

- Now Evaluate the expression $A + B = 2 + 3 = 5$ and push 5 into stack.

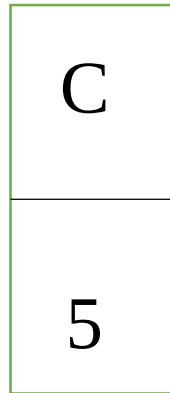
Stack



Cont...

Step 5 : Next element is operand C so push C into stack.

Stack



Cont...

Step 6 : Next element is operator $*$ so pop top 2 operands from the stack i.e C and 5.

Stack



Cont...

- Now Evaluate the expression $5 * C = 5 * 4 = 20$ and push 20 into stack.

Stack



Cont...

Step 7 : Next element is operand D so push D into stack.

Stack



Cont...

Step 8 : Next element is operator / so pop top 2 operands from the stack i.e D and 20.

Stack



Cont...

- Now Evaluate the expression $20 / D = 20 / 5 = 4$ and push 4 into stack.

Stack



Cont...

- Now final answer is 4.